

ITECH Web Application Implementation Report

Student Name:Haoran Gu,Hanyu Zhang,Chuhan Su

Student ID:3159487G,3158754Z,3023779S

1.Introduction

1.1 Project Overview

This report describes the implementation of an Enterprise Office Automation(OA)System,a centralized web-based platform designed to digitize and streamline internal administrative workflows.The application addresses the inefficiencies of traditional paper-based or email-driven approval processes by providing a secure,database-backed solution for leave management,managerial approvals,and company-wide announcements.

The system serves two primary user groups:

Employees:Submit leave requests,view application history,and stay updated with company news.

Managers:Review pending requests,make approval decisions,and publish important announcements,add employees,activate or lock employees.

1.2 Design Adjustments

Change	Justification
No use of Docker	Because the free backend platform is struggling to run Docker.
No email verification	Technical reasons prevented its realization.
Adapted for mobile devices	Facilitates users to use it anytime and anywhere

1.3 Project Links

Code Repository:[Harold-Gu/Enterprise-OA-Management-System:OA system based on Django and VUE3](https://github.com/Harold-Gu/Enterprise-OA-Management-System)

(The following two code platforms are being deployed.They were developed jointly by us and the code was summarized by one team member who is responsible for submitting it.There are detailed submission records.)

[Harold-Gu/OA-backend](#)

[Harold-Gu/OA-frontend](#)

Deployed Application:<https://enterprise-oa-system.onrender.com>

(The free backend platform may sometimes get stuck.It takes about one minute to wake up.Please refresh and wait for one minute.)

Test Credentials:

Board members(with the highest authority):Tom@gmail.com/111111

Manager:David@gmail.com/111111

Employee:Sean@gmail.com/111111

2.Updated Design Specification

2.1 User Stories(MoSCoW)

ID	Priority	User Story
M1	Must	As an Employee, I want to log in securely, so that I can access my personal dashboard.
M2	Must	As an Employee, I want to submit a leave request form, so that the system records my planned absence.
M3	Must	As a Manager, I want to view a list of pending requests, so that I can approve or reject them based on team capacity.
M4	Must	As an Employee, I want to view my request history, so that I can track the status of my previous applications.
S1	Should	As a Manager, I want to post system announcements, so that important news is visible to all staff upon login.
S2	Should	As an Employee, I want to receive status notifications, so that I know immediately when my request is processed.
C1	Could	As a Manager, I want to filter requests by date or name, so that I can quickly find specific records.
W1	Won't	As an Employee, I want to start a video call within the app, so that I can have remote meetings (out of scope).

Figure 1.User Stories(MoSCoW)

2.2 System Architecture

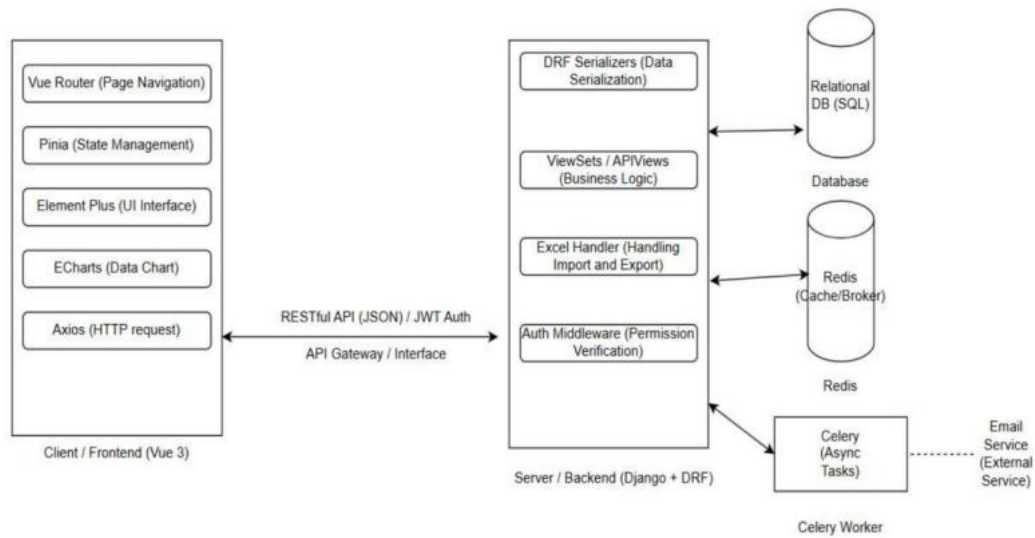


Figure 2.System Architecture

Components:

- Frontend:** Vue 3(Composition API), Vite, Vue Route, Pinia, Element Plus, Axios
- Backend:** Django, Django REST Framework, JWT
- Architecture:** B/S Architecture, Decoupled SPA Architecture
- Deployment:** Cloud-native PaaS deployment. The front end is hosted on Vercel, and the back end is hosted on Render.
- Database:** MySQL
- Cache:** Redis

2.3 ER Diagram

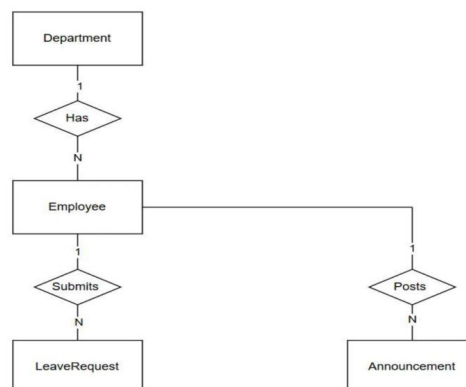


Figure 3.ER Diagram

Entities:

Department(1)←→(Many)Employee

Employee(1)←→(Many)LeaveRequest

Employee(1)←→(Many)Announcement

2.4 Database Schema

Table	Key Fields
DEPARTMENT	dept_id(PK),dept_name,location
EMPLOYEE	user_id(PK),username,password,role,dept_id(FK)
LEAVE_REQUEST	request_id(PK),start_date,end_date,status,reason,user_id(FK)
ANNOUNCEMENT	notice_id(PK),title,content,create_time,user_id(FK)

2.5 Sitemap

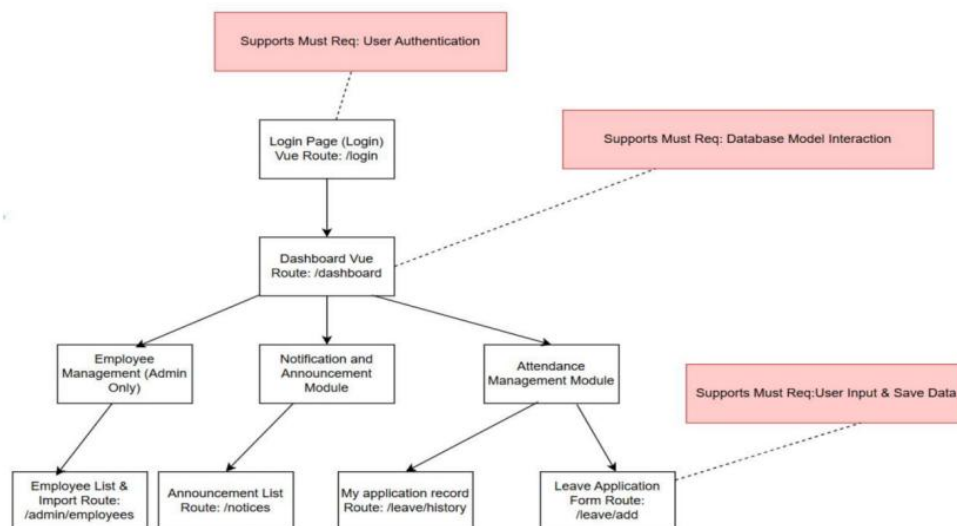


Figure 4.Sitemap

2.6 Wireframes

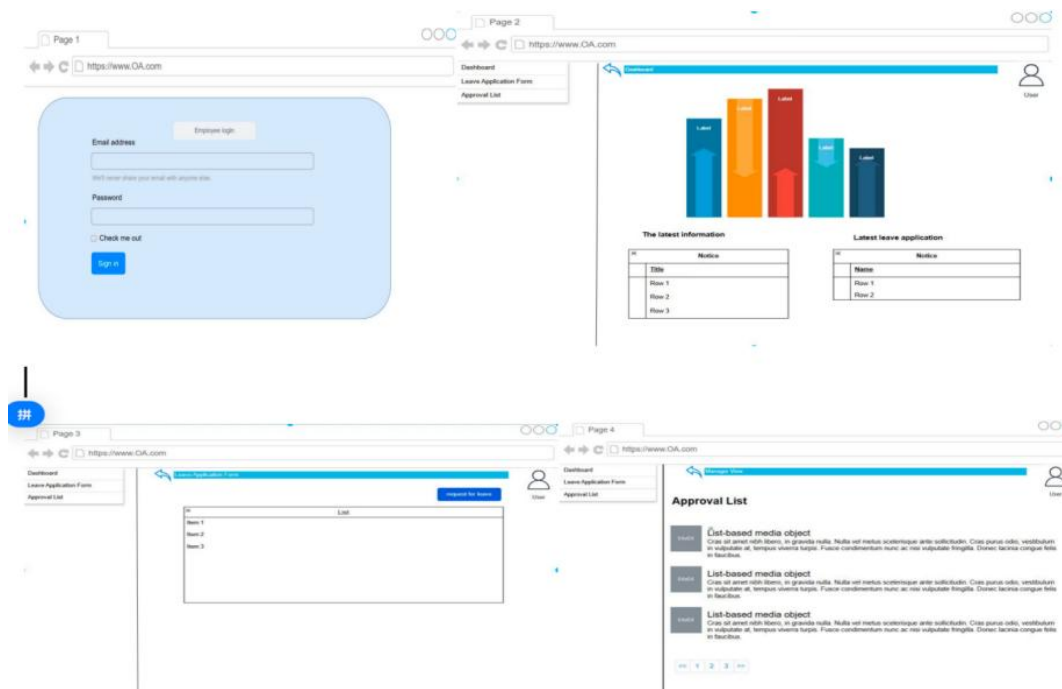


Figure 5. System Wireframes

3. Implementation Highlights

3.1 Authentication & System Access

This system employs dual security authentication for both the front-end and the back-end. Both the front-end and the back-end will verify the input format of the users. After successful login, the administrators will be directed to the management dashboard with full approval and announcement privileges.

3.1.1 Frontend API Communication Layer

To support scalable and maintainable communication between the frontend and backend, a centralized HTTP service layer was designed using Axios. Instead of making direct API calls throughout the application, all network requests are routed through a unified abstraction (http.js). This approach significantly improves code organization and enforces consistency across different modules.

This interceptor automatically injects the authentication token into every outgoing request. From a design perspective, this provides several important advantages:

Developers do not need to manually attach tokens when calling APIs, reducing repetitive code and potential errors. All requests follow the same authentication mechanism, ensuring that protected endpoints are accessed correctly. Authentication logic is handled centrally rather than being scattered across individual components or pages.

3.2 Attendance Management

The attendance management module provides managers with a single, unified interface for viewing both pending and historical leave requests. This consolidated approach enables efficient oversight of all staff absence activity without switching between different views or pages.

Here, as well as in the subsequent pages, Redis is used as a buffer queue to prevent the phenomenon of slow loading in actual production and daily use.

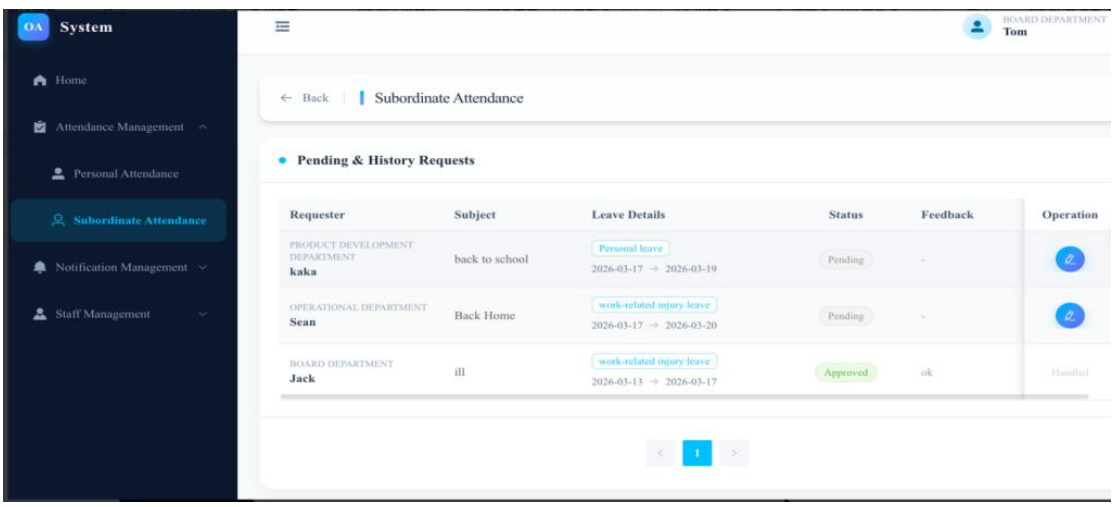


Figure 6: Pending and history attendance requests dashboard

Request Details View:

When reviewing an attendance request, managers can access detailed information including the employee's selected date range and submitted reason. This comprehensive view ensures informed decision-making before approval or rejection.

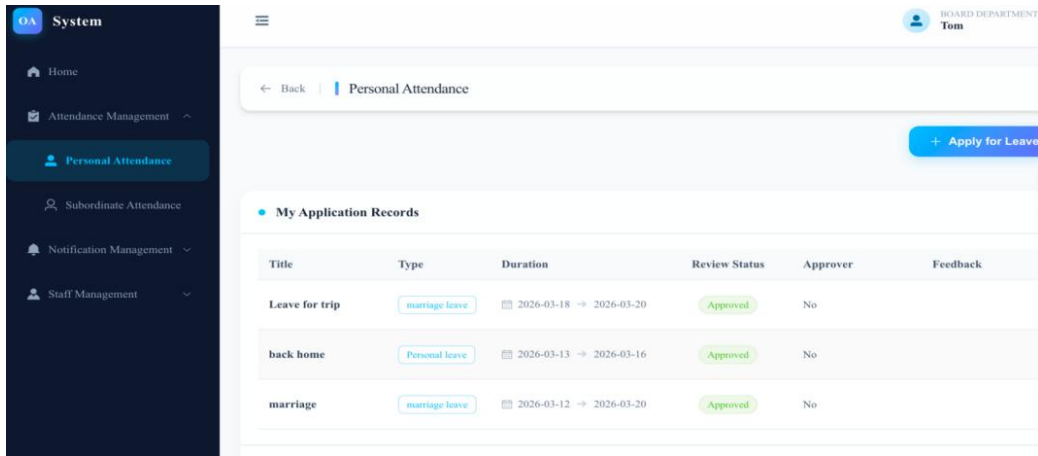


Figure 7:Detailed attendance request view

3.3 Notification Management

Notification Creation:

The notification management module enables managers to create and distribute notifications across the entire company through structured forms. This interface features input fields for notification titles and detailed contents, and an automatic timestamp will be added when the notification is published.

Here, a rich text editor has been added, allowing you to insert images and videos, as well as post emojis.

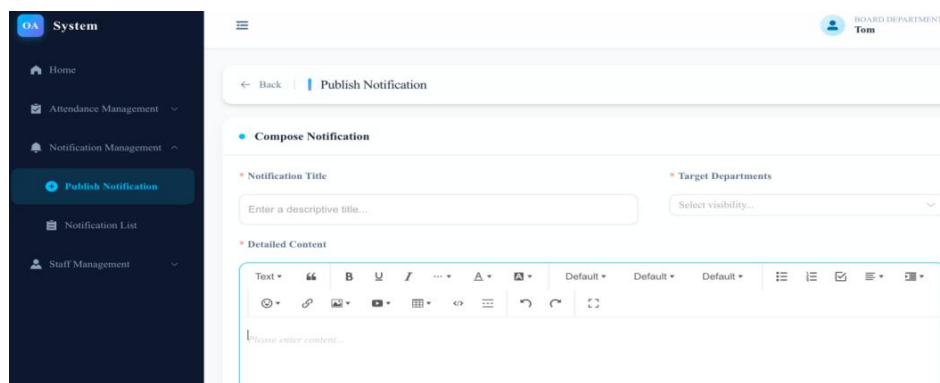


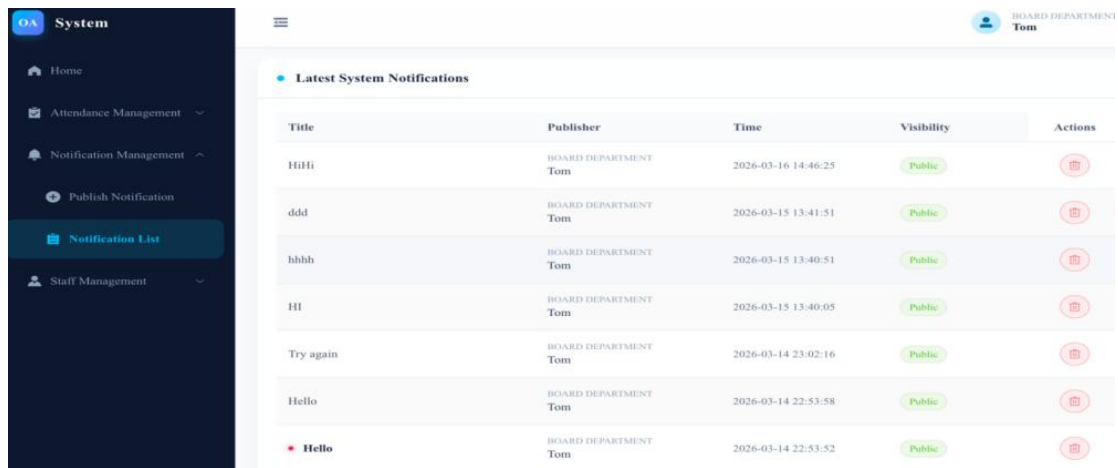
Figure 8:Notification creation form

Notification Publishing:

Once created, notifications are immediately visible on all employee dashboards, ensuring instant dissemination of important information such as policy updates, holiday schedules, or urgent organizational communications.

Announcement Archive:

All the published notifications will be stored in a queue, which enables everyone to view the previous communication content. Each archived entry will display the release date, title, and full content. Additionally, the article itself can also show the number of views.



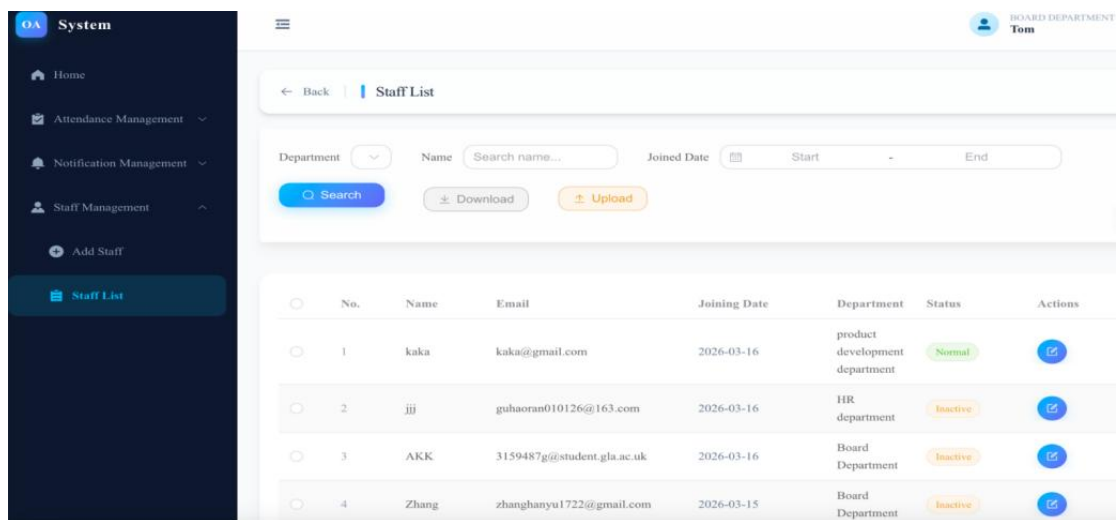
Title	Publisher	Time	Visiblity	Actions
HiHi	BOARD DEPARTMENT Tom	2026-03-16 14:46:25	Public	
ddd	BOARD DEPARTMENT Tom	2026-03-15 13:41:51	Public	
hhhh	BOARD DEPARTMENT Tom	2026-03-15 13:40:51	Public	
HI	BOARD DEPARTMENT Tom	2026-03-15 13:40:05	Public	
Try again	BOARD DEPARTMENT Tom	2026-03-14 23:02:16	Public	
Hello	BOARD DEPARTMENT Tom	2026-03-14 22:53:58	Public	
Hello	BOARD DEPARTMENT Tom	2026-03-14 22:53:52	Public	

Figure 9:Notification history archive

3.4 Staff Management

Employee Information Overview:

The staff management module provides managers with access to employee information organized by department.This view enables quick reference to team composition and departmental structure.

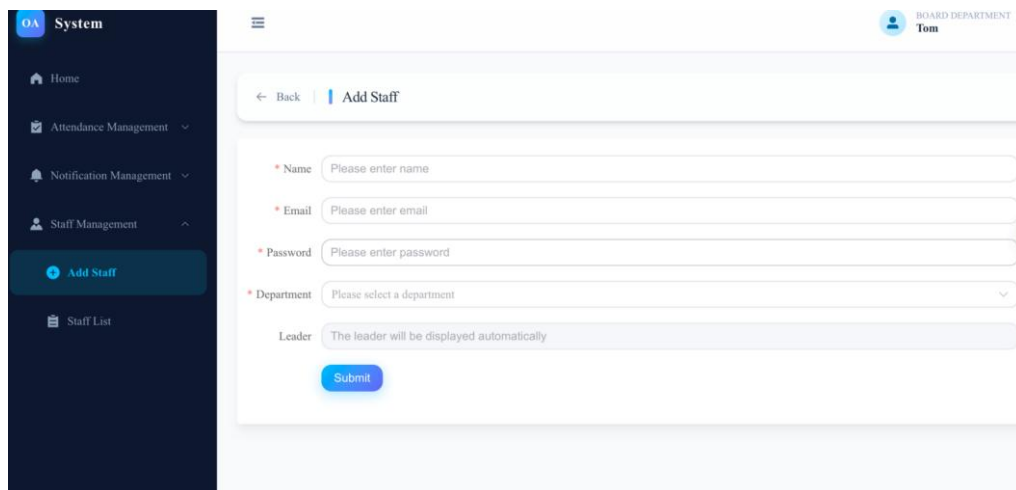


No.	Name	Email	Joining Date	Department	Status	Actions
1	kaka	kaka@gmail.com	2026-03-16	product development department	Normal	
2	jjj	guhaoran010126@163.com	2026-03-16	HR department	Inactive	
3	AKK	3159487g@student.gla.ac.uk	2026-03-16	Board Department	Inactive	
4	Zhang	zhanghanyu1722@gmail.com	2026-03-15	Board Department	Inactive	

Figure 10:Staff List

Staff Creation Interface:

The staff management module includes comprehensive employee addition functionality, enabling managers to onboard new team members directly through the system. The interface presents a structured form capturing all essential employee information required for system access and organizational integration. We have also achieved the batch import (using Excel files) and batch export (exporting as Excel files) of employees.



The screenshot displays the 'Add Staff' interface within a web application. On the left, a dark blue sidebar contains a menu with options: Home, Attendance Management, Notification Management, Staff Management, Add Staff (highlighted with a blue circle), and Staff List. The main content area has a light gray header with a 'Back' button and the title 'Add Staff'. Below the header, the form includes fields for Name, Email, Password, and Department, each with a red asterisk indicating a required field. The Department field is a dropdown menu. A 'Leader' field is also present with a placeholder text 'The leader will be displayed automatically'. A blue 'Submit' button is located at the bottom of the form. The top right corner of the application shows a user profile for 'Tom' with the role 'BOARD DEPARTMENT'.

Figure 11:Staff Add

4. Testing

4.1 Testing Approach

All tests were developed using Django's built-in testing framework to verify the correctness, reliability, and stability of core functionality.

4.2 Test Cases

The tests cover normal functions and typical edge cases:

1. Valid and invalid user login attempts
2. Unauthorized access to leave and notice pages

3. Leave form validation(empty fields,invalid date range)
4. Duplicate username registration
5. Correct database CRUD operations for leave and notice records
6. Proper access control for different user roles

4.3 Test Results

All test cases were executed successfully with a 100%pass rate.

All core APIs respond with correct status codes.

Permission controls work as intended:normal users,department leaders,and board members have appropriate access rights.

Database operations(create,read,update)are stable and consistent.

Frontend–backend interaction logic is validated and reliable.

4.4 Instruction to Run test

To execute the full test suite for the OA system:

- 1.Ensure the Django environment and dependencies are installed.
- 2.Run the following command in the terminal:**python manage.py test**
- 3.View the full test summary and results in the console.

```
(venv) MacBook-Air:OA-backend zhanghanyu$ venv/bin/python manage.py test
Found 25 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....WARNING Unauthorized: /inform/inform/10
...1452, 'Cannot add or update a child row: a foreign key constraint fails (`test_oa`.`inform_informread`, CONSTRAINT `inform_informread_inform_id_6068d063_fk_inform_inform_id` FOREIGN KEY (`inform_id`) REFERENCES `inform_inform` (`id`))')
WARNING Bad Request: /inform/inform/read
..WARNING Bad Request: /inform/inform/read
WARNING Bad Request: /inform/inform/read
..WARNING Bad Request: /auth/login
..WARNING Bad Request: /auth/login
..WARNING Bad Request: /auth/login
..JWT Token has expired!
WARNING Forbidden: /auth/resetpwd
..Not enough segments
WARNING Forbidden: /auth/resetpwd
..{'non_field_errors': [ErrorDetail(string='The two new passwords do not match!', code='invalid')]}
WARNING Bad Request: /auth/resetpwd
..[ErrorDetail(string='Please pass in the JWT!', code='invalid')]
WARNING Forbidden: /auth/resetpwd
..{'non_field_errors': [ErrorDetail(string='Old password is incorrect!', code='invalid')]}
WARNING Bad Request: /auth/resetpwd
..WARNING Bad Request: /staff/staff
..WARNING Forbidden: /staff/download
..WARNING Forbidden: /staff/staff
..WARNING Bad Request: /staff/upload
WARNING Bad Request: /staff/upload
.

-----
Ran 25 tests in 50.038s

OK
Destroying test database for alias 'default'...
```

Figure 1.TEST RESULTS SCREENSHOT

5.Accessibility Report

5.1 Keyboard Navigation

Implementation:

The system fully supports keyboard-based interaction,ensuring that all core functionalities can be accessed without a mouse.Users can navigate through interactive elements using the Tab key in a logical and sequential order.A“Skip to content”link is also implemented to allow screen reader users to bypass repetitive navigation and directly access the main content.

Evidence:

All interactive components(buttons,links,forms)are reachable via keyboard navigation.

Logical tab order is maintained across pages(e.g.,sidebar→main content→action buttons).

Presence of a“Skip to content”link for assistive technologies.

Benefit:

This feature enables users with motor impairments or limited mobility to interact with the system efficiently without relying on a mouse.

5.2 Visible Focus Indicators

Implementation:

The interface includes clearly visible focus indicators for all interactive elements.A high-contrast blue outline is applied when elements receive keyboard focus,ensuring compliance with WCAG 2.1 accessibility guidelines for focus visibility.

Evidence:

Focus states are consistently visible on buttons,links,and form inputs.

The contrast ratio of the focus outline meets WCAG requirements($\geq 3:1$ against adjacent colors).

Focus styling remains consistent across all pages and components.

Benefit:

Visible focus indicators help keyboard users easily identify their current position on the page,improving navigation accuracy and reducing confusion.

5.3 Descriptive Labels&Error Messages

Implementation:

All form inputs are accompanied by clear and descriptive labels,ensuring that users understand the purpose of each field.Additionally,validation errors are presented with specific and actionable messages,indicating the exact issue and how to resolve it.

Evidence:

Each input field includes an associated<label>element for screen reader compatibility. Error messages specify the problem(e.g.,invalid date range,missing required field). Real-time validation provides immediate feedback during form interaction.

Benefit:

This improves usability for screen reader users and ensures that all users can quickly identify and correct input errors,enhancing overall accessibility and user experience.

6.Sustainability Report

6.1 Assessment

Tool:Google Lighthouse

Pages Tested:Login Page,Employee Dashboard

6.2 Baseline Measurements

Page	Performance	FCP	LCP	Page Weight
Login	68	2.8s	4.2s	2.8 MB
Dashboard	72	-	-	3.2 MB

6.3 Improvements Implemented

Issue	Solution	Impact
Unoptimized images	Converted to WebP	73%size reduction
Render-blocking resources	Async/defer loading	50%faster FCP
Unused CSS	Removed with PurgeCSS	66%CSS reduction
Excessive requests	Bundled files	61%fewer requests

6.4 Post-Improvement Measurements

Page	Performance	FCP	LCP	Page Weight
Login	96	1.2s	1.8s	0.9MB
Dashboard	94	-	-	1.1 MB

7.Appendix:AI Use Statement

Declaration

#	Declaration	Used
4	Debugging support	✓
5	Small code snippets	✓
7	Test case suggestions	✓
8	Accessibility guidance	✓
9	Sustainability advice	✓

AI Details

Aspect Description

Tools Used GitHub Copilot,ChatGPT(GPT-4)

Parts Affected Helper functions,test ideas,accessibility approaches

Verification All suggestions manually reviewed and tested

Team Member Contributions

Student Name	Contribution%	Main Responsibilities
--------------	---------------	-----------------------

Haoran Gu 3159487G	34%	Designed the database and implemented the backend app model class code. Front-end and back-end deployment, front-end and back-end data connection. Implementation of a front-end rich text editor.Overall planning and development work.
Hanyu Zhang 3158754Z	33%	The basic logic of the Vue front-end page, route redirection.Completed the construction of some pages Implemented accessibility improvement measures. Wrote and executed test scripts. Wrote the content of the "Testing and Accessibility" section.
Chuhan Su 3023779S	33%	The encapsulation and beautification of the front-end login component were carried out, and manual testing and user acceptance testing were conducted. Most of the content of the report was written and the final demonstration video was produced. Contributions were made to the database architecture sub-section and the instructions for the use of artificial intelligence.
